FieldOps

🚀 Project Title

FieldOps AI – Voice-Driven Autonomous ERP for Field Service SMBs

Track Fit:

Primary: **AI for Knowledge Systems & Workers**
Secondary: **AI for Solopreneurs & SMBs**

🧠 Problem Context

Millions of blue-collar field service professionals — plumbers, electricians, AC technicians, repair contractors — operate in fast-paced, on-site environments where:

- They do not fill structured digital forms.
- They forget to update inventory.
- They delay invoice generation.
- They miss follow-up opportunities.
- They lose revenue due to poor documentation.
- They struggle with complex ERP systems.

Most ERP systems assume structured data entry.
However, field workers communicate naturally through voice notes — especially via mobile.

There is currently no intelligent system that converts messy, unstructured voice notes into real-time automated business execution.

This creates operational inefficiency and revenue leakage across the informal service economy.

🎯 Proposed Solution

FieldOps AI is a Zero-UI, Voice-Driven Autonomous ERP Agent that transforms a 30-second unstructured voice note into automated operational workflows.

Example Input:

> "Finished the Sharma job. Used 3 copper pipes, worked 2 hours. Heater is old — schedule follow-up in 6 months."

AI Automatically:

- Updates inventory (deducts 3 copper pipes)
- Calculates labor cost (2 hours)
- Generates invoice
- Schedules CRM follow-up
- Logs job summary
- Updates revenue dashboard

All without manual form filling.

This demonstrates true **Agentic AI** — not just text generation.

🧠 AI Depth & System Intelligence

FieldOps AI includes non-trivial AI implementation across multiple layers:

1️⃣ Multimodal Processing

Voice → Speech-to-Text

2️⃣ Structured Information Extraction

From messy natural speech, extract:

- Client name
- Materials used
- Quantity
- Labor time
- Follow-up intent
- Additional notes

3️⃣ Intent Classification

Detect operational actions:

- Inventory update
- Billing action
- CRM scheduling

4️⃣ Agentic Tool Calling

AI dynamically selects and executes backend functions:

- update_inventory()
- generate_invoice()
- create_followup()

5️⃣ Stateful Database Updates

System reflects real changes in:

- Inventory table
- Invoice records
- CRM tasks
- Revenue dashboard

This demonstrates structured reasoning + multi-step workflow execution.

🛠️ AI Tools & Technology Stack

🎙️ Speech-to-Text

- **OpenAI Whisper API** (free credits)
  OR
- Faster-Whisper (local open-source)

Used for accurate transcription of noisy voice notes.

🧠 Large Language Model (LLM)

- **GPT-4o-mini (OpenAI)** using free credits
  OR
- Mixtral / LLaMA 3 via Groq (free tier)

Used for:

- Structured JSON extraction
- Intent detection
- Tool selection
- Multi-step reasoning

🤖 Agent Orchestration

- OpenAI Function Calling (native tool-calling)
  OR
- LangChain Agent (open-source)

Enables LLM to trigger backend operational tools.

🔧 Backend

- FastAPI
- Pydantic (schema validation)
- SQLite / PostgreSQL (Supabase free tier)

📊 Frontend

- React / Next.js
- Tailwind CSS
- Live dashboards:
  - Inventory
  - Invoices
  - CRM tasks
  - Revenue analytics

🚀 Deployment

- Render (Backend – free tier)
- Vercel (Frontend)
- Supabase (Database)

Fully deployable within hackathon timeframe.

📊 Alignment with Judging Criteria

Depth of AI (15%)

- Structured extraction
- Agentic tool calling
- Multi-step workflow automation
- Stateful updates

Demo (20%)

- Play voice note
- Show live system updates in multiple dashboards

Business Viability (15%)

- Clear SaaS subscription model
- Large underserved market
- High ROI for users

Deployment (10%)

- Live working hosted product

UX (15%)

- Zero-form interaction
- Voice-first workflow
- Clean operational dashboards

💼 Target Audience

Primary:

- Independent plumbers
- Electricians
- Repair technicians
- Small contractors

Secondary:

- Small service agencies (2–20 employees)

Future Expansion:

- Logistics teams
- Field sales agents
- Construction crews

🔁 Retention Strategy

To ensure recurring usage:

- Weekly revenue summaries
- Inventory low-stock alerts
- Follow-up reminders
- Monthly performance analytics
- AI-driven business insights

Creates a habit-forming operational loop.

💰 Business Model

Freemium:

- Basic voice logging

Pro Tier:

- Auto invoicing + CRM

Premium:

- Analytics + tax-ready reports

Expansion:

- WhatsApp voice integration
- Regional language support
- Predictive revenue forecasting

🌍 Real-World Impact

- Reduces revenue leakage

- Saves 1–2 hours daily per worker
- Improves operational discipline
- Digitizes informal economy
- Increases follow-up conversion rates

At scale, this enhances productivity across millions of small service providers.

🎤 Final Pitch Line

FieldOps AI transforms messy voice notes into automated business operations, turning speech into structured revenue.

High-Level-Flow

🧠 FieldOps AI – High Level System Architecture

🔁 Core Flow (What happens when a user sends a voice note)

1️⃣ Voice Input Layer

User speaks a messy voice note:

> "Finished Sharma job, used 3 pipes, worked 2 hours…"

Components:

- Mobile mic / web mic
- Audio upload API (FastAPI endpoint)

Output: raw audio file

2️⃣ Speech → Text (ASR)

Audio is sent to **Whisper API**

Output:

"Finished Sharma job. Used 3 copper pipes. Worked 2 hours. Heater is old, follow up in 6 months."

3️⃣ AI Understanding Layer (Brain)

This is where your **real AI depth** comes.

Use **GPT-4o-mini with function calling** to:

Extract structured data

- Customer name → Sharma

- Materials used → 3 copper pipes
- Labor time → 2 hours
- Job status → completed
- Follow-up → 6 months
- Service type → heater repair

Classify intent

- Job logging
- Inventory usage
- CRM follow-up
- Invoice generation

Output (JSON):

```
{
 "customer": "Sharma",
 "job_status": "completed",
 "materials": [{"item": "copper pipe", "qty": 3}],
 "labor_hours": 2,
 "follow_up_days": 180,
 "actions": ["update_inventory", "create_invoice", "schedule_followup"]
}
```

4️⃣ Agentic Orchestration Layer

This is your **autonomous ERP engine**.

A FastAPI “Agent Controller” reads the JSON and calls tools:

Action	Tool Called
Update inventory	inventory_service()
Generate invoice	billing_service()
Calculate labor cost	pricing_engine()

Schedule follow-up	crm_service()
Log job summary	jobs_table.insert()
Update dashboard	analytics_service()

This makes it **not a chatbot → but a workflow executor**.

5️⃣ Database Layer (State of the Business)

Use **Supabase / SQLite**

Tables:

- customers
- jobs
- inventory
- invoices
- followups
- revenue_logs

Every voice note **mutates real business state**.

6️⃣ Real-Time Dashboard (UI Layer)

React / Next.js dashboard shows:

- Today's revenue
- Pending follow-ups
- Low stock alerts
- Recent jobs
- Monthly performance

No forms. Only:
🎤 “Speak → Done → Dashboard updates”

7️⃣ Retention Engine (Habit Loop)

System auto-generates:

- 📊 Weekly revenue summary
- 🔔 Low inventory alerts
- ⏰ Follow-up reminders
- 📈 Monthly performance report

These get sent via:

- In-app notifications
- (Future: WhatsApp)

This creates **operational dependency** → **retention**.

🧱 Architecture Diagram (Mental Model)

```
[Voice Input]
    ↓
[Whisper ASR]
    ↓
[GPT Structured Extraction + Intent Classification]
    ↓
[Agent Controller - FastAPI]
    ↓
[Tool Functions]
  ├── Inventory Service
  ├── Billing Engine
  ├── CRM Scheduler
  ├── Job Logger
  └── Analytics Updater
    ↓
[Database - Supabase]
    ↓
```

[Real-Time Dashboard - Next.js]
↓
[Retention Engine - Alerts & Summaries]

⚡ What Your System Can Do (Simple Demo Language)

You should explain it like this to judges:

👉 “A technician just speaks for 20 seconds.
Our AI understands the job, updates inventory, creates an invoice, schedules a follow-up, and updates revenue — automatically.”

In one voice note it can:

- Log a completed job
- Reduce inventory stock
- Calculate labor cost
- Generate invoice
- Schedule future service
- Update analytics dashboard

Zero typing. Zero forms.

🧪 Demo Flow (Use this exact story)

1. Show empty dashboard
2. Speak voice note
3. Show transcription
4. Show extracted JSON (AI depth)

5. Click “Run Agent” (or auto-run)
6. Dashboard updates live:
    - Revenue increased
    - Inventory reduced
    - Follow-up added
    - Invoice generated

This hits:

- Demo
- AI depth
- UX
- Deployment
- Business value

🧩 What to Build in 12 Hours (MVP Scope)

MUST HAVE

✅ Voice → text
✅ Structured extraction
✅ 3 tool calls (inventory, invoice, follow-up)
✅ Live dashboard
✅ Deployment

NICE TO HAVE

⭐ Weekly summary generator
⭐ Low stock alert
⭐ Multi-language voice

Role_division

👥 Team Split (Backend + AI)

🧠 Member 1 — AI Processing Pipeline (Voice → Structured JSON)

Owns: Everything up to “understanding the voice note”

Responsibilities

1. Audio upload API
2. Whisper speech-to-text integration
3. GPT structured extraction
4. Intent classification
5. Function-calling JSON output schema
6. Mock tool-call simulator (for local testing before integration)

Deliverables

- `/transcribe` → returns transcript
- `/extract` → takes transcript → returns structured JSON
- Final combined endpoint:

```
POST /process-voice
→ returns:
{
  transcript: "...",
  structured_data: {...},
  actions: [...]
}
```

JSON Contract (VERY IMPORTANT)

This is the **interface** to Member 2’s system:

```
{
```

```
 "customer_name": "Sharma",
 "job_status": "completed",
 "materials_used": [
   { "item": "copper pipe", "quantity": 3 }
 ],
 "labor_hours": 2,
 "service_type": "heater repair",
 "follow_up_days": 180,
 "actions": [
   "update_inventory",
   "create_invoice",
   "schedule_followup",
   "log_job"
 ]
}
```

Member 1 **must not** touch DB logic — only AI output.

⚙️ Member 2 — Agent Controller + Business Services

Owns: Everything after structured JSON

Responsibilities

1. Database schema (Supabase/SQLite)
2. Tool functions:
    - inventory_service()
    - billing_service()
    - crm_followup_service()
    - job_logger()
    - analytics_updater()
3. Agent Orchestrator endpoint:

```
POST /run-agent
→ input: structured JSON
→ executes tool calls
→ returns updated business state
```

4. Revenue + inventory calculations
5. Mock data for testing without AI

Deliverables

- `/run-agent` working with **hardcoded JSON**
- DB tables:
  - customers
  - inventory
  - jobs
  - invoices
  - followups
  - revenue_logs

Member 2 **must not** call Whisper/GPT — only consume JSON.

🔗 Integration Contract (The Plug Point)

Both systems connect via:

```
POST /run-agent
BODY = structured_data_from_AI
```

So final pipeline becomes:

```
/process-voice  → AI JSON
      ↓
/run-agent      → DB updates + business execution
```

This means:

- Both can build **in parallel**
- Both can test **with mocks**

- Integration is just **one API call**

🧪 Parallel Development Plan

Member 1 can test with:

Fake tool response:

{ "status": "tools executed successfully" }

Member 2 can test with:

Hardcoded JSON voice output:

{
 "customer_name": "Test User",
 "materials_used": [{"item": "pipe", "quantity": 2}],
 "labor_hours": 1,
 "follow_up_days": 30,
 "actions": ["update_inventory", "create_invoice"]
}

Schema

🧠1️⃣ AI Extraction Schema (LLM Output)

🎯 Purpose

Capture **semantic meaning from transcript** in the simplest, most LLM-friendly form.

⚠️ Not DB-safe
⚠️ Not used by frontend
⚠️ Only internal to AI pipeline

📦 Schema

```
{
 "customer_name": "string",
 "service_type": "string",
 "materials_mentioned": [
   {
     "item_name": "string",
     "quantity": "number"
   }
 ],
 "labor_mentioned": {
   "duration_text": "string"
 },
 "follow_up_mentioned": {
   "is_required": "boolean",
   "time_text": "string | null",
   "reason_text": "string | null"
 },
```

```
  "job_status_text": "string",
  "notes": "string",
  "intents": [
    "log_job",
    "update_inventory",
    "create_invoice",
    "schedule_followup"
  ]
}
```

🔍 Field Meaning

Field	Meaning
customer_name	Raw name as spoken
service_type	e.g., “heater repair”
materials_mentioned	Natural item names + qty
labor_mentioned.duration_text	e.g., “worked 2 hours”
follow_up_mentioned.time_text	e.g., “6 months”
job_status_text	“finished”, “pending”, etc.
notes	extra context

intents	actions inferred by LLM

🧠 Why This Exists

- Keeps GPT output simple
- Avoids numeric parsing errors
- Allows normalization layer to convert text → numbers

⚙️2️⃣ Execution Schema (Agent Contract) ⭐ CORE

🎯 Purpose

Deterministic, validated input for backend tools.

This is the **shared_schema** between Member 1 and Member 2.

📦 Schema

```
{
 "customer": {
   "name": "string",
   "phone": "string | null"
 },
 "job": {
   "service_type": "string",
   "status": "completed | pending",
```

```
  "notes": "string"
},
"labor": {
  "hours": "number",
  "rate_per_hour": "number | null"
},
"materials": [
  {
    "item": "string",
    "quantity": "number",
    "unit_cost": "number | null"
  }
],
"follow_up": {
  "required": "boolean",
  "after_days": "number | null",
  "reason": "string | null"
},
"invoice": {
  "generate": "boolean"
},
"actions": [
  "log_job",
  "update_inventory",
  "create_invoice",
  "schedule_followup",
```

```
  "update_analytics"
 ],
 "meta": {
  "source": "voice_note",
  "timestamp": "ISO-8601 string"
 }
}
```

🔍 Field Meaning

customer

Field	Required	Use
name	✅	find/create customer
phone	❌	future WhatsApp invoices

---

job

Field	Use
service_type	job categorization
status	determines job lifecycle

notes	stored in jobs table

---

labor

Field	Use
hours	labor cost calculation
rate_per_hour	if null → use default rate

---

materials[]

Each item:

- decrement inventory
- add invoice line item

---

follow_up

Field	Logic
required	if true → create CRM task
after_days	due_date = today + days

reason	CRM description

---

invoice

`generate = true` → billing service runs

---

actions

Agent **must only execute listed tools**
Prevents accidental operations.

---

meta

Used for:

- job.created_at
- analytics logs
- audit trail

🧠 Default Rules (Important)

If LLM misses data:

Field	Default
materials	[]
labor.hours	0

follow_up.required	false
invoice.generate	true if job completed
actions	must include log_job

🖥️3️⃣ Response Schema (Frontend/UI)

🎯 Purpose

Tell dashboard **what changed** — not how.

No AI fields. No DB structure.

📦 Schema

```
{
 "transcript": "string",
 "job_logged": "boolean",
 "inventory_updated": "boolean",
 "invoice_generated": "boolean",
 "followup_scheduled": "boolean",
 "revenue_added": "number",
 "low_stock_items": ["string"],
 "next_followup_date": "ISO-8601 string | null",
 "job_summary": {
   "customer_name": "string",
```

```
  "service_type": "string",
  "labor_hours": "number",
  "materials_used": [
    {
      "item": "string",
      "quantity": "number"
    }
  ]
 }
}
```

🔍 Field Meaning

Field	UI Usage
transcript	show in demo panel
job_logged	green check indicator
inventory_updated	stock update indicator
invoice_generated	billing confirmation
followup_scheduled	CRM confirmation
revenue_added	update revenue widget

low_stock_items	alert card
next_followup_date	CRM card
job_summary	recent activity feed

🔄 End-to-End Transformation Example

AI Extraction

“Worked 2 hours, used 3 pipes, follow up in 6 months”

⬇ normalized

Execution


```
{
 "customer": { "name": "Sharma", "phone": null },
 "job": { "service_type": "heater repair", "status": "completed", "notes": "Heater old" },
 "labor": { "hours": 2, "rate_per_hour": null },
 "materials": [{ "item": "copper pipe", "quantity": 3, "unit_cost": null }],
 "follow_up": { "required": true, "after_days": 180, "reason": "Old heater" },
 "invoice": { "generate": true },
 "actions": ["log_job","update_inventory","create_invoice","schedule_followup","update_analytics"],
 "meta": { "source": "voice_note", "timestamp": "2026-02-21T22:30:00Z" }
}
```


⬇ after execution

Response


```
{
 "transcript": "Finished Sharma job...",
 "job_logged": true,
 "inventory_updated": true,
 "invoice_generated": true,
 "followup_scheduled": true,
 "revenue_added": 2500,
 "low_stock_items": ["copper pipe"],
 "next_followup_date": "2026-08-20",
 "job_summary": {
   "customer_name": "Sharma",
   "service_type": "heater repair",
   "labor_hours": 2,
   "materials_used": [
     { "item": "copper pipe", "quantity": 3 }
   ]
 }
}
```

🧠 How This Shows “Real AI Depth” to Judges

You can say:

👉 “We separate semantic extraction, deterministic execution, and UI state into three schemas.

This ensures LLM outputs never directly mutate financial data and allows safe agentic automation.”

That signals **enterprise-grade thinking**.

🛡️ Validation Rules (Must Implement)

Execution schema must:

- Reject negative quantities
- Reject labor_hours < 0
- Require customer.name
- Require job.service_type
- Require meta.timestamp

If invalid → return safe error to UI.

🏁 Ownership Summary

Schema	Owner	Used By
AI Extraction	Member 1	AI pipeline only
Execution	Shared	Agent engine
Response	Member 2 → Frontend	UI

Project Structure

🧱 Full High-Level Project Structure

```
project/
│
├── backend/
│   ├── ai_pipeline/          ← Member 1
│   │   ├── ai_connector.py
│   │   ├── transcribe.py
│   │   ├── extract.py
│   │   └── prompts.py
│   │
│   ├── agent_engine/         ← Member 2
│   │   ├── agent_connector.py
│   │   ├── services.py
│   │   ├── logger.py         ← NEW (execution + state logs)
│   │   └── state_tracker.py  ← NEW (before/after snapshots)
│   │
│   ├── shared_schema/
│   │   ├── action_schema.py
│   │   ├── execution_schema.py
│   │   └── log_schema.py
│   │
│   ├── database/
│   │   ├── models.py
│   │   ├── db.py
│   │   └── seed_data.py
│   │
│   ├── api_gateway/
│   │   └── main_router.py    ← single UI-facing route
│   │
│   └── app.py
│
└── frontend/
    ├── services/
    │   └── apiClient.ts      ← talks to backend
    │
    ├── features/
    │   ├── voiceInput/
    │   ├── dashboard/
    │   ├── demoPanel/        ← shows AI + execution logs
    │   └── agentTraceViewer/ ← NEW (beautiful state transitions)
    │
    └── pages/
        └── index.tsx
```

🔌 Backend Connector Contracts

🧠 AI Connector (Member 1)

📄 `backend/ai_pipeline/ai_connector.py`

Function

process_voice(audio_file: UploadFile) -> dict

Returns

```
{
 "structured_data": { ...shared_schema },
 "transcript": "string"
}
```

✔ Uses Whisper
✔ Uses GPT structured extraction
✔ Enforces JSON schema
✔ No DB access
✔ No business logic

⚙️ Agent Connector (Member 2)

📄 `backend/agent_engine/agent_connector.py`

Function

run_agent(structured_data: dict) -> dict

Returns (Enhanced With Logs)

```
{
 "execution": {
   "job_logged": true,
   "inventory_updated": true,
   "invoice_generated": true,
   "followup_scheduled": true,
   "revenue_added": 2500,
   "low_stock_items": ["copper pipe"],
   "next_followup_date": "2026-08-20"
 },
 "agent_trace": [
```

```
  {
    "step": "INVENTORY_UPDATE",
    "before": {"copper pipe": 100},
    "after": {"copper pipe": 97}
  },
  {
    "step": "INVOICE_CREATED",
    "amount": 2500
  }
 ]
}
```


✔ Deterministic
✔ Executes only actions[]
✔ Logs every state change
✔ Returns execution summary

🗂️ New Logging Layer (VERY IMPORTANT)

📄 `backend/agent_engine/logger.py`

Function

log_step(step_name: str, before: dict, after: dict)

Stores:

- step_name
- before_state (JSON)
- after_state (JSON)
- timestamp

📄 `backend/agent_engine/state_tracker.py`

Handles:

- Capturing DB state before mutation
- Capturing DB state after mutation
- Passing snapshots to logger

This enables:

✅ “100 → 97” style transitions
✅ Audit trail
✅ Beautiful demo logs

🔗 API Gateway (Single Frontend Endpoint)

📄 `backend/api_gateway/main_router.py`

Route Exposed to Frontend

POST /api/process-voice
Content-Type: multipart/form-data
Body: audio file

Internal Flow

audio
→ process_voice()
→ run_agent()
→ merged response

Response Sent to Frontend (Enhanced)

```
{
 "transcript": "Finished Sharma job...",
 "structured_data": { ... },     ← optional for demoPanel
 "execution": {
   "job_logged": true,
   "inventory_updated": true,
   "invoice_generated": true,
   "followup_scheduled": true,
```

```
  "revenue_added": 2500,
  "low_stock_items": ["copper pipe"],
  "next_followup_date": "2026-08-20"
 },
 "agent_trace": [
  {
   "step": "INVENTORY_UPDATE",
   "before": {"copper pipe": 100},
   "after": {"copper pipe": 97}
  },
  {
   "step": "INVOICE_CREATED",
   "amount": 2500
  },
  {
   "step": "FOLLOWUP_SCHEDULED",
   "due_date": "2026-08-20"
  }
 ]
}
```


This powers:

- Dashboard updates
- DemoPanel
- AgentTraceViewer

🌐 FastAPI Entry

📄 `backend/app.py`

Exposes:

GET /health
POST /api/process-voice
POST /api/process-text ← demo safety endpoint

Nothing else required for hackathon demo.

🎯 Frontend Integration Design

Frontend never talks to AI or Agent separately.

Only calls:

POST /api/process-voice

🧩 Frontend Service Layer

📄 `frontend/services/apiClient.ts`

Function

processVoice(audioFile: File): Promise<ProcessVoiceResponse>

Updated Response Type

```
type ProcessVoiceResponse = {
 transcript: string;
 structured_data?: any;
 execution: {
   job_logged: boolean;
   inventory_updated: boolean;
   invoice_generated: boolean;
   followup_scheduled: boolean;
   revenue_added: number;
   low_stock_items: string[];
   next_followup_date: string | null;
 };
 agent_trace: {
   step: string;
   before?: any;
   after?: any;
   amount?: number;
   due_date?: string;
 }[];
};
```

🖥️ Frontend Feature Responsibilities

🎤 voiceInput/

- Record audio
- Send to apiClient.processVoice()
- Show loading state
- Handle fallback to process-text

📊 dashboard/

Updates after response:

- Revenue added today
- Low stock alert
- Next follow-up card
- Recent job log

🧪 demoPanel/ (VERY IMPORTANT FOR JUDGES)

Shows:

- Transcript
- Extracted JSON
- Execution results
- Agent trace logs (step-by-step)
- Before → After transitions

This proves:

AI reasoning
State mutation

Agentic workflow
Auditability

🔄 End-to-End Flow (With Logging)

User speaks
→ Frontend records audio
→ POST /api/process-voice
→ AI Connector (speech + structured extraction)
→ Agent Connector executes tools
→ Logger records each state change
→ Response includes execution + agent_trace
→ Dashboard updates
→ DemoPanel shows full trace

🛡️ Stability Rules for Hackathon

- Frontend uses only one main endpoint
- Backend connectors expose only one function each
- Shared schema frozen early
- Structured JSON strictly validated
- State tracking wrapped in try/except
- Add mock mode if AI fails

⚡ Optional Demo Safety Endpoint

POST /api/process-text
Body: { text: "Finished Sharma job..." }

Same response format as `/process-voice`.

Prevents mic/demo failure.

🏁 What Each Team Can Build in Parallel

Team Member	Can Work Without Others
Member 1 (AI)	Yes — returns structured JSON
Member 2 (Agent)	Yes — consumes mock JSON
Frontend	Yes — uses mock API until ready

Zero blocking.

backend

🧱 High-Level Backend Folder Structure

```
backend/
│
├── ai_pipeline/            ← Member 1 owns
│   └── ai_connector.py       ← exports: process_voice()
│
├── agent_engine/           ← Member 2 owns
│   └── agent_connector.py    ← exports: run_agent()
│
├── shared_schema/          ← common contract (read-only after lock)
│
├── api_gateway/            ← final integration layer
│   └── main_router.py        ← calls AI → Agent
│
└── app.py                  ← FastAPI entry point
```

🔌 Connector Responsibilities

🧠 `ai_pipeline/ai_connector.py` (Member 1)

Must expose **one function only**:

process_voice(audio_file) → structured_json

Internally it can:

- call Whisper
- call GPT
- validate schema

But **output must match shared_schema**.

No DB calls. No business logic.

⚙️ `agent_engine/agent_connector.py` (Member 2)

Must expose **one function only**:

run_agent(structured_json) → execution_result

Internally it can:

- update DB
- calculate invoice
- update inventory
- schedule follow-up

But it **must not** call AI APIs.

🔗 Final Integration Flow

`api_gateway/main_router.py`

This is the only place where both meet:

audio → process_voice() → structured_json → run_agent() → response

So if either member finishes early, they just plug into this.

📜 Shared Contract Folder

shared_schema/

Contains:

- JSON schema definition
- (later) Pydantic model

Rule:
❗ Both members can read
❗ Only edit together once
❗ Lock after Hour 3

🚀 FastAPI Entry Point

app.py

Responsibilities:

- Create FastAPI app
- Register `main_router`
- Health check route `/health`

No business logic here.

🧩 Ownership Summary

Folder	Owner	Touch Policy
ai_pipeline	Member 1	Only they edit
agent_engine	Member 2	Only they edit
shared_schema	Both (early only)	Lock after schema freeze
api_gateway	Both (integration phase)	Joint work
app.py	Whoever integrates	Minimal changes

🏁 Build Order

Phase 1 (Parallel)

- Member 1 → builds `ai_pipeline`
- Member 2 → builds `agent_engine`

- Both agree on `shared_schema`

Phase 2 (Integration)

- Wire in `main_router.py`
- Test end-to-end

🛡️ Anti-Breakage Rule

If schema changes:

- Update `shared_schema`
- Both update connectors
- Never hot-fix inside routers

This keeps demo stable.

If you want next, I'll give you:

- The exact **function signatures**
- FastAPI **route layout**
- Minimal **health + test endpoints**
- Mock mode for offline demo safety

Ayush

Good. This is the right moment to decide.

Short answer:

👉 **Yes — you can lock this structure.**
But I will make **2 small hackathon-focused refinements** to reduce risk and speed you up.

You are 10-hour constrained.
Optimization > architectural purity.

🧠 Overall Verdict on Your Structure

Your structure is:

✔ Clean
✔ Professionally layered
✔ Correct separation of AI vs Agent
✔ Has normalization layer (very strong)
✔ Has shared contract
✔ Has logging separation

Architecturally: 9.5/10

For hackathon speed: we simplify slightly.

🚨 Minor Improvements (Hackathon Optimization)

We do NOT change design philosophy.

We simplify complexity slightly.

🔧1️⃣Merge validation into normalization

Right now you have:

normalization/

validation/

For hackathon, merge them.

Instead of:

normalization/

validation/

Do:

normalization/

  normalizer.py

  duration_parser.py

  labor_parser.py

  action_resolver.py

  execution_validator.py

Keep validation inside normalization layer.

Less jumping between folders.
Faster debugging.

🔧2️⃣ Remove ai_logger.py (Optional)

Unless you are logging AI-specific debug traces,
you don't need a separate AI logger.

Hackathon rule:

If it's not demo-visible, don't overbuild it.

Keep:

retry_handler.py

Remove AI-only logger unless necessary.

🎯 Final Hackathon-Optimized Structure

Here is the slightly refined version I recommend locking:

```
project/
│
├── backend/
│
│   ├── ai_pipeline/                ← MEMBER 1
│   │
│   │   ├── ai_connector.py
│   │
│   │   ├── speech/
│   │   │   └── whisper_service.py
│   │
│   │   ├── extraction/
│   │   │   ├── gpt_service.py
│   │   │   ├── prompt_builder.py
│   │   │   └── extraction_schema.py
│   │
│   │   ├── normalization/
│   │   │   ├── normalizer.py
│   │   │   ├── duration_parser.py
│   │   │   ├── labor_parser.py
│   │   │   ├── action_resolver.py
```

```
│   │   │   └── execution_validator.py   ← moved here
│   │
│   │   └── utils/
│   │       └── retry_handler.py
│   │
│   ├── agent_engine/                   ← MEMBER 2
│   │   ├── agent_connector.py
│   │   ├── services.py
│   │   ├── logger.py
│   │   └── state_tracker.py
│   │
│   ├── shared_schema/
│   │   ├── execution_schema.py
│   │   └── log_schema.py
│   │
│   ├── database/
│   │   ├── models.py
│   │   ├── db.py
│   │   └── seed_data.py
│   │
│   ├── api_gateway/
│   │   └── main_router.py
│   │
│   └── app.py
│
└── frontend/
```

```
├── services/
│   └── apiClient.ts
│
├── features/
│   ├── voiceInput/
│   ├── dashboard/
│   ├── demoPanel/
│   └── agentTraceViewer/
│
└── pages/
    └── index.tsx
```

This is what I recommend locking.

🧠 Why This Is The Sweet Spot

Because:

- Still layered
- Still clean separation
- Still schema-aware
- Still professional
- Less fragmentation
- Faster to implement
- Easier to debug under pressure

🚀 Important: What NOT To Add Now

Do NOT add:

❌ Additional abstraction layers
❌ Extra middlewares
❌ Complex dependency injection
❌ Over-engineered service classes
❌ Multiple routers

Hackathon = controlled minimalism.

🎯 From Member 1 POV

You now have:

- Clear AI extraction layer
- Clear normalization layer
- Clear execution schema contract
- Clear single entry function

That's perfect.

🏆 Final Answer

Yes — lock this refined version and start building.

You are at the correct architecture maturity level.

Now we move from architecture → implementation.

Next step, tell me:

Do you want to implement first:

1️⃣ Whisper integration
2️⃣ GPT extraction prompt
3️⃣ Normalization logic
4️⃣ Execution schema Pydantic model

We'll build it step by step. 🚀

Nandani

🧠 Design Principles for `agent_engine/`

Your execution layer must:

1. Separate orchestration from business logic
2. Separate services from DB models
3. Separate validation from execution
4. Support mock mode
5. Log everything cleanly
6. Be tool-pluggable
7. Be schema-driven
8. Be testable without AI
9. Avoid circular imports
10. Be extendable (future: WhatsApp, analytics AI, forecasting)

🧱 Final Production-Ready Folder Structure

`backend/agent_engine/`

```
agent_engine/
│
├── agent_connector.py
│
├── orchestrator/
│   ├── agent_controller.py
│   ├── action_dispatcher.py
│   └── execution_context.py
│
```

```
├── validation/
│   ├── execution_validator.py
│   ├── business_rules.py
│   └── normalization.py
│
├── services/
│   ├── inventory/
│   │   ├── inventory_service.py
│   │   └── inventory_repository.py
│   │
│   ├── billing/
│   │   ├── billing_service.py
│   │   ├── pricing_engine.py
│   │   └── invoice_repository.py
│   │
│   ├── crm/
│   │   ├── followup_service.py
│   │   └── followup_repository.py
│   │
│   ├── jobs/
│   │   ├── job_service.py
│   │   └── job_repository.py
│   │
│   ├── analytics/
│   │   ├── analytics_service.py
│   │   └── revenue_repository.py
│   │
│   └── customer/
│       ├── customer_service.py
│       └── customer_repository.py
│
├── state/
│   ├── state_tracker.py
│   └── state_snapshot.py
│
├── logging/
│   ├── execution_logger.py
│   ├── audit_logger.py
│   └── log_formatter.py
│
├── config/
```

```
│   ├── agent_settings.py
│   └── defaults.py
│
├── exceptions/
│   ├── agent_exceptions.py
│   └── validation_exceptions.py
│
└── utils/
    ├── date_utils.py
    ├── idempotency.py
    └── money_utils.py
```

Now I'll explain why this structure is industry-grade.

🔌 Top-Level Entry

`agent_connector.py`

This is the only exported entry point.

```
def run_agent(execution_schema: dict) -> dict:
```

Responsibilities:

- Validate input
- Create execution context
- Call orchestrator
- Return response schema
- Catch safe errors

This file should be VERY small.

Think of it as your public API.

🧠 Orchestrator Layer

`orchestrator/`

This is the brain of deterministic execution.

`agent_controller.py`

- Main control loop
- Reads `actions[]`
- Calls dispatcher
- Tracks execution results

`action_dispatcher.py`

Maps action string → service function

Example:

```
ACTION_MAP = {
    "update_inventory": inventory_service.update_inventory,
    "create_invoice": billing_service.generate_invoice,
    "schedule_followup": followup_service.schedule_followup,
}
```

This makes adding a new action extremely easy.

`execution_context.py`

Holds:

- execution_schema
- db session
- accumulated results
- revenue delta
- state snapshots

This prevents passing 10 arguments everywhere.

🛡️ Validation Layer

`validation/`

You must never trust LLM output.

`execution_validator.py`

- Validate required fields
- Reject negative numbers
- Ensure timestamp exists
- Enforce enum values

`business_rules.py`

- If job.status == completed → invoice.generate = True
- If follow_up.required == False → ignore after_days
- If labor.hours == 0 → no labor charge

`normalization.py`

- Convert None to defaults
- Ensure empty materials = []
- Convert floats safely

This keeps execution deterministic.

🧰 Services Layer (Modular + Scalable)

Each service:

- Contains business logic
- Calls its repository
- Does NOT know about AI
- Does NOT know about orchestrator

This is clean architecture.

Example: Inventory

`inventory_service.py`

- Decrement stock
- Check low stock
- Return affected items

`inventory_repository.py`

- Pure DB operations
- SQLAlchemy / Supabase logic

This separation makes unit testing easy.

🧾 Billing Layer

`billing_service.py`

- Calls pricing_engine
- Creates invoice

- Returns revenue_added

`pricing_engine.py`

- Calculate labor cost
- Add material costs
- Apply tax (future-ready)

Keep pricing isolated so future features are easy.

👤 Customer Service

Customer creation must happen BEFORE other services.

`customer_service.py`

- find_or_create_customer()

This avoids repeating DB logic in every service.

📊 Analytics

`analytics_service.py`

- Log revenue
- Update metrics
- Prepare dashboard numbers

Future:

- Add forecasting AI here.

🔍 State Tracking

`state_tracker.py`

- Capture before snapshot
- Capture after snapshot
- Track which tables changed

Used for:

- Demo trace viewer
- Debugging
- Audit log

This will impress judges.

📝 Logging Layer

`execution_logger.py`

- Structured logs
- Each action logged with status

`audit_logger.py`

- Persist execution history

Important for production thinking.

⚙️ Config Layer

`agent_settings.py`

- Default labor rate
- Low stock threshold
- Tax rate

`defaults.py`

- Fallback invoice behavior
- Default business rules

Never hardcode constants inside services.

🚨 Exceptions Layer

Create custom exceptions:

```
class ValidationError(Exception)
class BusinessRuleError(Exception)
class ToolExecutionError(Exception)
```

Never throw raw Python errors to UI.

🧩 Utils Layer

- `date_utils.py` → follow-up due date calculation
- `money_utils.py` → safe decimal operations
- `idempotency.py` → prevent duplicate execution (important if retried)

🔥 Why This Structure Wins

Benefit	Why It Matters
Modular services	Easy to extend
Dispatcher map	Add tools instantly
Validation layer	Prevent AI corruption
Context object	Clean orchestration
Repository pattern	Easy DB swap
Config isolation	Production ready
State tracker	Beautiful demo trace
Logging layer	Debug friendly

🚀 How Adding a New Feature Looks

Example: Add `send_whatsapp_invoice`

1. Create:

```
services/notifications/whatsapp_service.py
```

2. Add action:

```
"send_whatsapp_invoice": whatsapp_service.send_invoice
```

No other file touched.

That's scalable architecture.

🧠 Real Industry Pattern You're Following

This is basically:

- Clean Architecture
- Service Layer Pattern
- Repository Pattern
- Orchestrator Pattern
- Contract-Driven Design

You are building this like a SaaS startup backend.

🏁 MVP Version (12-Hour Optimized)

If time gets tight:

You can temporarily merge:

- repository + service
- remove audit logger
- simplify analytics

But keep the folder layout.

Structure > shortcuts.

Importance_Of_three Schemas

🧠 The 3-Schema Design

1️⃣ AI Extraction Schema (LLM Output Schema)

👉 Used **inside the AI pipeline only**
👉 Optimized for GPT reliability, not DB

Purpose

Convert messy transcript → clean structured meaning

Contains

- customer name (raw string)
- service type (raw phrase)
- materials (natural names)
- labor hours
- follow-up intent
- action list
- free-text notes

⚠️ No DB IDs
⚠️ No calculated fields
⚠️ No pricing logic

This keeps GPT output **simple and stable**.

2️⃣ Execution Schema (Agent Input Schema) ⭐ CORE CONTRACT

👉 This is the **shared_schema** between Member 1 and Member 2
👉 Optimized for deterministic backend execution

Purpose

Tell the agent **exactly what to do** with validated fields

Contains

- normalized customer object
- materials with quantities
- labor hours
- boolean switches for actions
- follow-up days
- invoice flag
- timestamp metadata

This schema:

- drives tool calling
- maps directly to DB operations
- prevents AI hallucination from breaking logic

This is your **main contract**.

3️⃣ Response Schema (UI/Dashboard Schema)

👉 Used **after execution**
👉 Optimized for frontend rendering

Purpose

Tell the UI what changed in the business

Contains

- job_logged: true/false
- revenue_added: number

- low_stock_items: []
- next_followup_date
- invoice_generated
- inventory_updated

⚠️ No raw AI fields
⚠️ No internal DB structure

Only **user-facing outcomes**.

🔄 Data Flow Across Schemas

Voice
→ Transcript
→ AI Extraction Schema
→ Execution Schema (validated + normalized)
→ Agent runs tools
→ Response Schema
→ Dashboard

Each stage **reduces ambiguity** and **increases determinism**.

Mem -1 checklist

✅ PHASE 1 — Core AI Pipeline Setup

🎙️ 1. Whisper Integration (Speech → Text)

File: `speech/whisper_service.py`

Checklist:

- Install OpenAI SDK
- Load API key from env
- Implement `transcribe(audio_file)`
- Ensure it supports `.wav` and `.mp3`
- Return plain string (no JSON)
- Add basic error handling
- Test with 3 sample audio files
- Handle empty transcript edge case

Goal:

transcript: str

🧠 2. GPT Extraction Layer

Files:

- `extraction/gpt_service.py`
- `extraction/prompt_builder.py`

Checklist:

- Write strict system prompt
- Enforce JSON-only output
- Enforce AI Extraction Schema format
- Set temperature = 0
- Implement `extract_structured(transcript)`
- Parse JSON safely
- Catch malformed JSON
- Retry once if invalid

Goal:
Return valid `AIExtractionSchema`.

📦 3. AI Extraction Schema Validation

File: `extraction_schema.py`

Checklist:

- Define nested Pydantic models
- Add default empty lists where needed
- Add optional fields properly
- Validate GPT output into model
- Test invalid GPT output handling

Goal:
Never allow invalid LLM structure to pass forward.

✅ PHASE 2 — Normalization Layer (Critical)

🔄 4. Normalizer Core

File: `normalization/normalizer.py`

Checklist:

- Implement `normalize(extraction: AIExtractionSchema)`
- Map customer_name → customer.name
- Map status_text → completed/pending
- Map service_type directly
- Transfer notes safely

Goal:
Produce partially built ExecutionSchema.

⏳ 5. Duration Parser

File: `duration_parser.py`

Checklist:

- Convert "6 months" → 180
- Convert "2 weeks" → 14
- Convert "10 days" → 10
- Handle null safely
- Return int | None

Goal:

after_days: int | None

🕑 6. Labor Parser

File: `labor_parser.py`

Checklist:

- Convert "2 hours" → 2
- Convert "1.5 hours" → 1.5
- Handle null → 0
- Strip extra words
- Ensure numeric output

Goal:

hours: float

🧠 7. Action Resolver

File: `action_resolver.py`

Checklist:

- Always include "log_job"
- If materials exist → add "update_inventory"
- If labor > 0 OR materials exist → add "create_invoice"
- If follow_up.required → add "schedule_followup"
- Always add "update_analytics"
- Remove duplicates
- Return clean list

Goal:
Deterministic `actions[]`.

📏 8. Execution Schema Validation

File: `execution_validator.py`

Checklist:

- Import ExecutionSchema from shared_schema
- Validate final object
- Enforce required fields
- Enforce enums
- Throw clean error if invalid

Goal:
Never send bad data to Member 2.

✅ PHASE 3 — AI Connector Orchestration

🔗 9. ai_connector.py

This is your master file.

Checklist:

- Implement `process_voice(audio_file)`
- Call whisper_service
- Call gpt_service
- Validate extraction schema
- Normalize
- Validate execution schema
- Return:

```
{
  "transcript": "...",
  "structured_data": ExecutionSchema
}
```

- Add try/except wrapper

- Log errors cleanly

This is your deliverable function.

✅ PHASE 4 — Stability & Edge Cases

🧪 10. Test Edge Cases

Test manually with transcripts:

- No materials mentioned
- No labor mentioned
- Only follow-up mentioned
- No follow-up
- Ambiguous status
- Empty voice note
- Very short transcript
- Very long transcript

🔁 11. Add Retry Handler

File: `retry_handler.py`

Checklist:

- If GPT JSON parse fails → retry once
- If still fails → raise structured error
- Avoid infinite loops

🛡️ 12. Add Safe Defaults

Ensure:

- materials default = []
- labor.hours default = 0
- follow_up.required default = False
- invoice.generate computed logically
- meta.timestamp = server time (NOT GPT)

✅ PHASE 5 — Integration Readiness

🔌 13. Prepare Clean Return Object

Ensure final return is:

```
{
  "transcript": "...",
  "structured_data": {...execution_schema}
}
```

Nothing extra.
No debug noise.
No internal schema.

🧪 14. Test With Mock Agent

Before real integration:

- Send structured_data to dummy function
- Confirm format matches shared_schema
- Confirm no missing fields

Mem2

🗄️1️⃣ Best Database for This Use Case

✅ Best Fit: Supabase (PostgreSQL)

Why:

- Managed PostgreSQL (production-grade)
- Free tier available
- SQL relational integrity (important for ERP)
- Row-level security ready
- Easy dashboard to inspect data live
- Real-time features (future)
- Easy deployment

You are building:

- Customers
- Jobs
- Inventory
- Invoices
- Followups
- Revenue logs

This is relational business data → PostgreSQL is correct.

SQLite is fine for local dev, but for demo credibility:

👉 Use Supabase PostgreSQL as final.

Recommended setup:

- Dev: SQLite (fast iteration)
- Demo: Supabase Postgres

🧠2️⃣ End-to-End Build Strategy

You should build in **6 Controlled Phases**.

🚀 PHASE 0 — Environment Setup (Before Coding Logic)

Goal:

Stable foundation before touching business logic.

Tasks:

- Create virtual environment
- Install:
  - fastapi
  - sqlalchemy
  - psycopg2 (or asyncpg)
  - pydantic
  - alembic (optional)
- Setup:
  - config/.env
  - database/db.py
  - DB session factory

- Setup logging framework

🧪 TEST POINT 0

- Start FastAPI
- Hit `/health`
- Ensure DB connection test endpoint works

You must verify:

- DB connects successfully
- App boots without circular imports

Do NOT continue if this fails.

🏗️ PHASE 1 — Database Schema Design

Do NOT write services yet.

First define your database models.

Tables:

- customers
- inventory
- jobs
- invoices
- invoice_items
- followups
- revenue_logs

Design Principles

- Use foreign keys
- Use numeric for money (Decimal)
- Add created_at, updated_at
- Add soft delete optional

🧪 TEST POINT 1 — DB Integrity

After creating models:

1. Run migrations / create tables
2. Manually insert test records
3. Query them
4. Ensure relationships work

Test:

- Create customer
- Create job referencing customer
- Create invoice referencing job
- Query join relationships

You should confirm:

- No FK errors
- Cascade behavior works as expected

Only after this → move ahead.

⚙️ PHASE 2 — Repository Layer (DB Access Layer)

Now create repository classes.

Example:

```
customer_repository.py
inventory_repository.py
invoice_repository.py
```

Responsibilities:

- Pure DB operations
- No business logic
- No pricing
- No validation

🧪 TEST POINT 2 — Repository Unit Tests

Test WITHOUT services:

- find_or_create_customer()
- decrement_inventory()
- create_invoice()
- insert_followup()

Use hardcoded test JSON.

You must verify:

- Inventory reduces correctly
- Invoice row created correctly

- Follow-up due date stored properly
- Revenue log entry works

If this fails → fix here.
Never debug DB through orchestrator.

🧮 PHASE 3 — Business Services Layer

Now build:

- inventory_service
- billing_service
- crm_service
- job_service
- analytics_service

Each should:

- Call repository
- Apply business rules
- Return structured result

Example:
inventory_service returns:

```
{
  "inventory_updated": True,
  "low_stock_items": []
}
```

🧪 TEST POINT 3 — Service Layer Tests

Mock execution_schema manually.

Test:

- If materials empty → inventory untouched
- If labor 0 → no labor cost
- If follow_up.required False → no CRM row
- If job.status completed → invoice generated

Test edge cases:

- Negative quantity (should error)
- Missing customer (should error)

You must simulate bad input here.

This protects you from AI errors.

🧠 PHASE 4 — Validation + Normalization

Now implement:

- execution_validator.py
- business_rules.py
- normalization.py

Validation must:

- Reject negative numbers
- Require customer.name
- Require job.service_type
- Require meta.timestamp

Normalization must:

- Default materials = []
- Default labor.hours = 0
- Default follow_up.required = False

🧪 TEST POINT 4 — Corrupt Input Tests

Feed:

- Missing timestamp
- Negative quantity
- Invalid action
- Invalid job status

Ensure:

- System returns safe error
- DB remains untouched

This is CRITICAL.

🧠 PHASE 5 — Orchestrator + Dispatcher

Now wire:

agent_connector → validation → orchestrator → dispatcher → services

---

🧪 TEST POINT 5 — Partial Execution Safety

Test:

Case 1:
actions = ["log_job"]

Only job table changes.

Case 2:
actions = ["update_inventory","create_invoice"]

Both run.

Case 3:
Unknown action

Should error safely.

Test idempotency:
Run same execution twice.
Ensure duplicate invoices are not created.

🗄️ WHEN TO CONNECT SUPABASE

Only after:

- All local SQLite tests pass
- All services work
- No validation bugs

Then:

1. Create Supabase project
2. Copy connection string
3. Replace DB_URL
4. Run migrations
5. Seed data

🧪 TEST POINT 6 — Supabase Integration

Run:

- One complete execution
- Verify rows in Supabase dashboard
- Check foreign keys
- Check numeric fields
- Check timestamps

Ensure no environment mismatch issues.

🔁 PHASE 6 — Integration With Member 1

Now test with real structured JSON.

Test flow:

AI → execution_schema → run_agent → DB updates → response_schema

---

🧪 TEST POINT 7 — End-to-End Test

Simulate:

Voice note example.

Verify:

- Job logged
- Inventory reduced
- Invoice created
- Revenue added
- Follow-up scheduled

- Response schema matches UI contract

🛡️ Production-Ready Practices You MUST Follow

1️⃣ Never Trust Input

Always validate.

2️⃣ Use Transactions

Wrap run_agent in DB transaction.
If one tool fails → rollback everything.

3️⃣ Structured Logging

Log:

- timestamp
- action
- result
- errors

4️⃣ Idempotency Key

Use meta.timestamp + customer name.
Prevent duplicate runs.

5️⃣ Decimal for Money

Never use float for money.

6️⃣ Environment Config Isolation

No hardcoded DB strings.

7️⃣ Graceful Error Handling

Return:

```
{
  "error": true,
  "message": "Validation failed"
}
```

Never crash.

🧪 Final Demo Checklist

Before demo:

- Clean DB
- Seed inventory with low stock scenario
- Test voice flow once
- Test duplicate submission
- Test follow-up date logic
- Test dashboard reflects revenue